Démo - Recette (GH03)

Abel FARES, Yan RABEFANIRAKA

15 Janvier 2025

Contents

1 Tests des arguments **1**
1.1 Arguments d'affichage . . . 1
1.2 Multiples arguments . . . 2
1.3 Fichier inexistant . . . 2
1.4 Mauvaise extension (*Décompression*) . . . 2

2 Sur du texte (ASCII) **2**
2.1 Exemple du sujet . . . 3
2.2 Texte long . . . 3

3 Autres fichiers **3**
3.1 Codage utf-8 . . . 3
3.2 Image . . . 4

On a constitué notre démo en plusieurs fichiers *bash*.

1 Tests des arguments

L'une des caractéristiques essentielles du cahier des charges est la prise en compte des arguments par l'application. Voici comment on a choisi de montrer qu'ils fonctionnent grâce au fichier:

1.1 Arguments d'affichage

Les arguments "`-b`" et "`--bavard`" permettent d'afficher des informations sur le processus de l'application. Il s'agit de l'argument par défaut, mais on peut aussi utiliser "`-s`" ou "`--silencieux`" pour ne pas afficher quoi que ce soit.

- Test quand aucun argument n'est donné:

```
# Pour la compression
./compression tests/exemple_demo.txt
# Pour la décompression
./decompression results/exemple_demo.txt.hff
```

- Test pour les arguments "`-b`" et "`--bavard`":

```
# Pour l'argument "-b" pour la compression
./compression -b tests/exemple_demo.txt
# Pour l'argument "--bavard" pour la compression
./compression --bavard tests/exemple_demo.txt

# Pour l'argument "-b" pour la décompression
./decompression -b results/exemple_demo.txt.hff
```

```
# Pour l'argument "--bavard" pour la décompression
./decompression --bavard results/exemple_demo.txt.hff
```

- Test pour les arguments "`-s`" et "`-silencieux`" pour la compression:

```
# Pour l'argument "-s" pour la compression
./compression -s tests/exemple_demo.txt
# Pour l'argument "--silencieux" pour la compression
./compression --silencieux tests/exemple_demo.txt
```

```
# Pour l'argument "-s" pour la décompression
./decompression -s results/exemple_demo.txt.hff
# Pour l'argument "--silencieux" pour la décompression
./decompression --silencieux results/exemple_demo.txt.hff
```

1.2 Multiples arguments

Pour de multiiples arguments, le programme est supposé uniquement prendre en compte le dernier argument en compte.

- Test pour de multiples arguments en compression:

```
# Pour l'argument définitif -s
./compression -b -s -b -s tests/arguments_demo.txt
# Pour l'argument définitif -b
./compression -s -b -s -b tests/arguments_demo.txt
```

- Test pour de multiples arguments en décompression:

```
# Pour l'argument définitif -s
./decompression -b -s -b -s results/arguments_demo.txt.hff
# Pour l'argument définitif -b
./decompression -s -b -s -b results/arguments_demo.txt.hff
```

1.3 Fichier inexistant

Si le fichier donné en argument n'existe pas, on a choisi de renvoyer une erreur.

- Test pour des fichiers inexistants:

```
# Pour la compression
./compression -b tests/fichier_non_existant.txt

# Pour la décompression
./decompression -b results/fichier_non_existant.hff
```

1.4 Mauvaise extension (*Décompression*)

Pour la `Decompression`, le programme ne peut pas ouvrir d'autres fichiers que ceux compressés avec notre programme *i.e.* ceux avec l'extension `.hff`.

- Test pour une mauvaise extension pour décompression

```
./decompression -b results/mauvaise_extension.hfd
```

2 Sur du texte (ASCII)

Avec des petits tests différents, on peut notamment vérifier que la sortie de `Compression` est bien un fichier binaire plus petit que l'original, et que `Décompression` renvoie le même résultat. On peut aussi vérifier l'affichage demandé dans le sujet de cette manière.

2.1 Exemple du sujet

- Le test du sujet est très court mais est donc assez efficace pour vérifier l'arbre généré, ainsi que les symboles codés qui en découlent. Cela permet aussi de vérfifier facilement que 'Decompression' retransmet le bon arbre et écrit le même fichier d'origine:

```
./compression -b tests/texte_sujet.txt
```

On peut vérifier que le résultat est bien un fichier binaire avec `xxd`:

```
xxd -b results/texte_sujet.txt.hff
```

et pour les tailles de fichier:

```
# Pour le fichier d'origine
stat -c %s tests/texte_sujet.txt

# Pour le résultat
stat -c %s results/texte_sujet.txt.hff
```

Pour obtenir le résultat décompressé:

```
./decompression -b results/texte_sujet.txt.hff
```

On peut vérifier que le fichier décompressé est bien le même que le fichier d'origine:

```
diff tests/texte_sujet.txt results/texte_sujet.txt.hff
```

On peut aussi en lire le contenu:

```
xxd -b results/texte_sujet.txt.hff
```

2.2 Texte long

- Ce texte est un Lorem Ipsum. Il permet surtout de confirmer que le programme marche pour des fichiers plus conséquents:

```
./compression -b tests/texte_long.txt
```

,puis:

```
./decompression -b results/texte_long.txt.hff
```

3 Autres fichiers

Notre programme doit fonctionner pour n'importe quel fichier binaire. Ce n'est pas encore entièrement le cas puisque même si la compression renverra un fichier binaire compressé, il ne correspondra pas forcément à l'original. On peut tout de même montrer que le programme s'exécute correctement pour du texte qu'importe le codage.

3.1 Codage utf-8

- On peut tester notre application avec d'autres codages, comme le utf-8 par exemple:

```
./compression -b tests/texte_sujet.utf8.converted
```

puis:

```
./decompression -b results/texte_sujet.utf8.converted.hff
```

On voit que le fichier décompressé est bien le même que le fichier d'origine:

```
diff tests/texte_sujet.utf8.converted results/texte_sujet.utf8.converted.hff.d
```

Si on utilise un symbole aux codages différents comme le "é" qui n'existe pas en ASCII, on a aussi ce qui suit:

```
./compression -b tests/texte_e_act.txt
```

puis:

```
./decompression -b results/texte_e_act.txt.hff
```

3.2 Image

- On peut tester avec l'image `texte_sujet_comp.png` utilisée pour le rapport:

```
./compression -b tests/texte_sujet_comp.png
```

puis:

```
./decompression -b results/texte_sujet_comp.png.hff
```

On voit effectivement que ce ne sont pas les mêmes fichiers:

```
diff tests/texte_sujet_comp.png results/texte_sujet_comp.png.hff.d
```